ETL Pipeline Example: Airflow Extraction Task

Understand how to orchestrate ETL pipelines using Apache Airflow by integrating a Python extraction function as a task within a DAG. Learn to use Airflow operators, especially PythonOperator, to automate and schedule data extraction from production databases, verifying task success and preparing for subsequent ETL steps.

After completing the previous task, we now have a function called `extract` for extracting the latest batch of data from the production database in a file called `helper.py`. We'll add this function as a task in an airflow DAG to complete the extract step.

This is how we build our pipelines using airflow, by adding more and more tasks to a particular DAG.

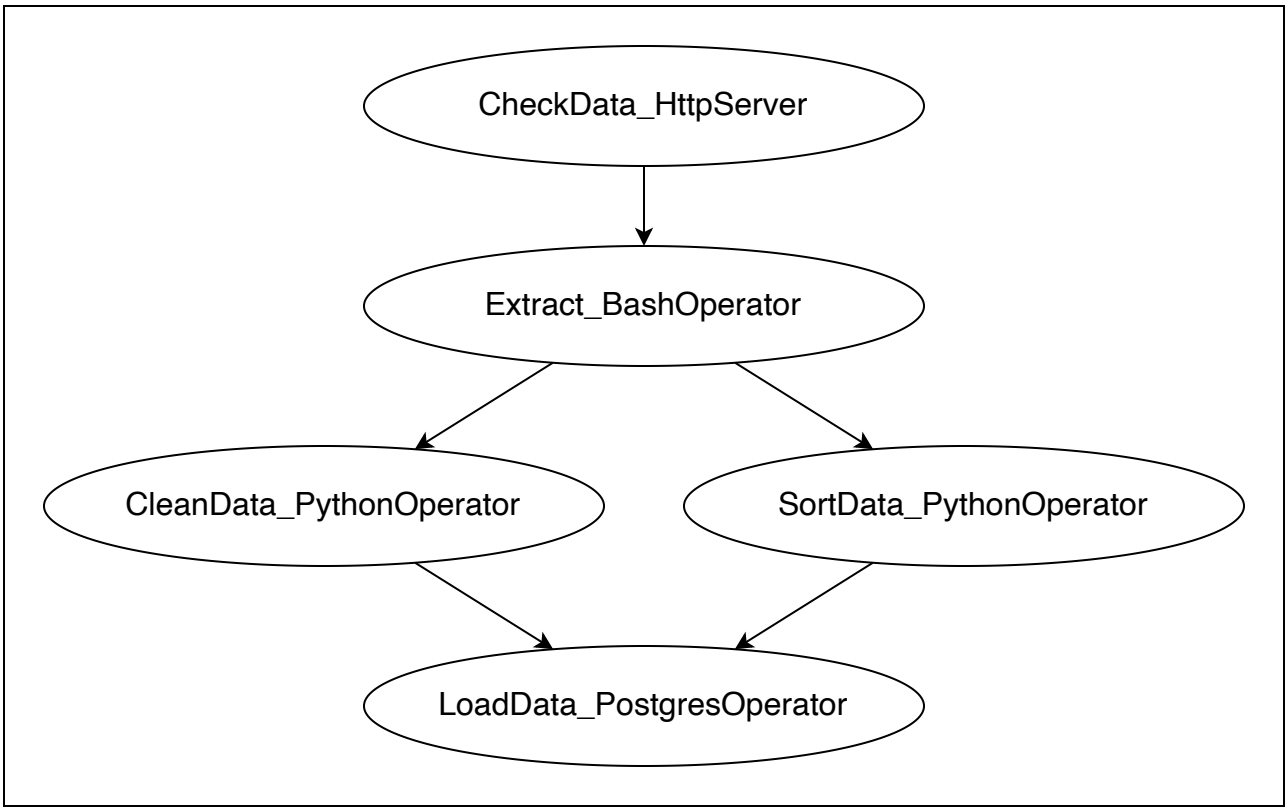
Airflow operators

Before showing how to do that, let's discuss airflow operators. In Airflow, operators are the building blocks that define tasks in a pipeline. Each operator is different and can be used to perform a particular type of task, such as executing a SQL query, transferring files, running a Python script, etc. Some common operators are:

- `BashOperator`: Used to execute Bash commands or a script
- `PythonOperator`: Used to execute Python commands or a script
- `HttpSensor`: Pokes an HTTP service until a response is received
- `PostgresOperator`: Used to interact with a PostgreSQL database

Ask

Each task in an Airflow DAG is executed by some type of operator. An ETL pipeline can be built by using a combination of different operators working in sequence or parallel. For example, an ETL pipeline built using Airflow operators can look like this:



ETL pipeline as an Airflow DAG

In our example, we've created the `extract` function using Python. In order to use this Python script in Airflow, we need to execute it using the `PythonOperator`.

Let's see how we can use the `PythonOperator` to add the `extract` function to a DAG.

Adding the `extract` function to a DAG

First, to create an Airflow DAG, we need to create a directory called `dags` a
`/airflow/dags`. Airflow will automatically search for Python files in this directory.
The DAG will be a Python file called `social_media_pipeline.py` and will import

Ask

the `DAG` module from the `airflow` package. We'll also import the `extract` function we wrote earlier from the file `helper.py`.

Then, we'll execute this function using a `PythonOperator` inside the newly created DAG object.

social_media_pipe... Python

```python
from airflow import DAG
from airflow.operators.python import PythonOperator

from helper import extract # importing the extract function from helper.p
import datetime

# Create a DAG object
with DAG(
    dag_id = 'social_media_pipeline', # DAG name
    start_date=datetime.datetime(2023,8,29), # start date
    schedule=datetime.timedelta(hours=4) # schedule(run every 4 hours)
    ) as dag:
```

Run

Creating an airflow DAG and adding the extract function

The DAG object takes in three parameters:

1. `dag_id`: This is the unique identifier for the DAG.
2. `start_date`: This is the date from which the DAG should start running.
3. `schedule_interval`: How often should the DAG run? The company requested that the pipeline will run every four hours. Therefore, `schedule_interval` is equal to `datetime.timedelta(hours=4)`.

We can test if the task we added to the DAG is working properly using the command `airflow tasks test <DagName> <TaskName> <ExecutionDat` Ask our case, the command to check if the new task works is as follows:

```
airflow tasks test social_media_pipeline extract 2023-08-29
```

Run the command in the following terminal widget to check if the task works. If everything works properly and the `extract` task succeeds, we should have a CSV file called `social_media_data.csv`.

We can verify that by using the `ls` command. The content of `social_media_data.csv` can be viewed using the following command:

```
cat social_media_data.csv | column -t -s, | less -S
```

Terminal 1

Terminal

Loading...

With that, we’ve finished the first task in the ETL pipeline. We extracted the relevant social media data from our company’s production database. In the next step of the pipeline, we’ll write a script to transform the data according to the business demands.

Ask

Mark As Completed

Ask